

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC: CƠ SỞ LẬP TRÌNH

ĐỒ ÁN MÔN HỌC

Halo - Cyber Access Engine

Giảng viên lý thuyết: Trương Toàn Thịnh
Giảng viên thực hành: Nguyễn Trọng Việt

Sinh viên thực hiện: Mai Thúc Hải Đăng - 24120276

Thành phố Hồ Chí Minh, tháng 5/2026

Mục lục

1	Đánh giá mức hoàn thành đề án	2
2	Thiết kế hệ thống	2
2.1	Cấu trúc dữ liệu	2
2.2	Tiền xử lý	7
2.3	Thống kê top 10 tài nguyên	11
3	Cách sử dụng	13
4	Các khó khăn gặp phải và Hướng giải quyết	13

1. Đánh giá mức hoàn thành đề án

- Đã hoàn thành 100% các yêu cầu giữa kì.
- Tự thiết kế cấu trúc dữ liệu với mảng động, không sử dụng các container có sẵn của thư viện.
- Đọc và tiền xử lí, lưu trữ trên bộ nhớ Heap thành công và có thu hồi bộ nhớ cấp phát động khi kết thúc chương trình.
- Thực hiện đầy đủ, chính xác 3 chức năng khai thác hệ thống:
 1. Liệt kê hành trình truy xuất tài nguyên Device → App → Resource của 1 User trong 1 khoảng thời gian.
 2. Liệt kê hành trình User → Device → App của 1 Resource trong 1 khoảng thời gian.
 3. Thống kê Top 10 Resource được truy xuất nhiều nhất trong khoảng thời gian cho trước.

2. Thiết kế hệ thống

2.1. Cấu trúc dữ liệu

Để đảm bảo tốc độ tải dữ liệu nhanh và tiết kiệm tối đa bộ nhớ, hệ thống đã load toàn bộ file vào một mảng ký tự duy nhất trên vùng nhớ Heap với ký tự được lưu dưới dạng const char* để tối ưu thời gian đọc).

Listing 1: Cấu trúc dữ liệu DataArray

```
1 struct DataArray
2 {
3     char*** rows;
4     char** fileDataArr;
5     char* fileData;
6     string* columnNames;
7     int columnSize;
8     int rowSize;
9 };
```

Trong đó:

- `fileData` là một chuỗi chứa toàn bộ kí tự của file.
- `fileDataArr` là mảng 1 chiều chứa data nhưng chia ra theo từng cột từng dòng bằng cách các phần tử trong `fileDataArr` chứa con trỏ trỏ đến đầu các từ của `fileData`, và các từ đó được ngăn cách bằng việc gán các dấu `,` và `\n` bằng `'\0'`, khi đó để giải phóng bộ nhớ thì chỉ cần `delete[] fileData` thôi.
- `rows` là mảng 2 chiều chứa data theo dạng từng dòng từng cột bằng cách mỗi khi duyệt tới `\n` thì sẽ gán `rows` ở dòng tương ứng bằng `fileDataArr[currentRow * cols + currentCol]`.
- `columnName` là mảng 1 chiều chứa các string tương ứng là tên của các cột ở đầu file CSV, nhằm tăng tính linh hoạt cho bất kì file CSV nào thỏa mãn, chỉ cần bổ sung logic về sau thôi.

Cụ thể quá trình đọc file được em xử lí trong hàm sau:

Listing 2: Hàm `loadData`

```
1 int loadData(const string& filename, dataArray& datas)
2 {
3     ifstream file(filename);
4     if (!file.is_open())
5     {
6         cout << "Khong the mo file" << filename << endl;
7         return 0;
8     }
9
10    file.seekg(0, ios::end);
11    size_t fileSize = file.tellg();
12    file.seekg(0, ios::beg);
13    datas.fileData = new char[fileSize + 1];
14    file.read(datas.fileData, fileSize);
15    size_t actualSize = file.gcount();
16    datas.fileData[actualSize] = '\0';
17    file.close();
18
19    if (!actualSize)
20    {
21        return 0;
```

```
22     }
23
24     int rows = 0;
25     int cols = 1;
26     size_t i = 0;
27
28     for (; i < actualSize; ++i)
29     {
30         if (datas.fileData[i] == ',')
31         {
32             ++cols;
33         }
34         else if (datas.fileData[i] == '\n')
35         {
36             ++rows;
37             ++i;
38             break;
39         }
40     }
41
42     for (; i < actualSize; ++i)
43     {
44         if (datas.fileData[i] == '\n')
45         {
46             ++rows;
47         }
48     }
49
50     if (datas.fileData[actualSize - 1] != '\n')
51     {
52         ++rows;
53     }
54
55     datas.initColumns(cols);
56     if (--rows <= 0)
57     {
58         return 0;
59     }
60
61     datas.initRows(rows);
```

```
62  datas.fileDataArr = new char* [rows * cols];
63  int currentCol = 0;
64  char* currentWord = datas.fileData;
65
66  for (i = 0; i < actualSize; ++i)
67  {
68      bool isComma = (datas.fileData[i] == ',');
69      bool isNewline = (datas.fileData[i] == '\n');
70
71      if (isComma || isNewline)
72      {
73          datas.fileData[i] = '\0';
74          datas.setColumnName(currentCol, string(currentWord));
75          ++currentCol;
76          currentWord = &datas.fileData[i + 1];
77
78          if (isNewline)
79          {
80              ++i;
81              break;
82          }
83      }
84  }
85
86  int currentRow = 0;
87  currentCol = 0;
88
89  int locIdx = datas.getColumnIndex("location");
90  int timeIdx = datas.getColumnIndex("timestamp");
91
92  for (; i < actualSize; ++i)
93  {
94      bool isComma = (datas.fileData[i] == ',');
95      bool isNewline = (datas.fileData[i] == '\n');
96
97      if (isComma || isNewline)
98      {
99          datas.fileData[i] = '\0';
100          if (currentCol < cols)
101          {
```

```
102     datas.fileDataArr[currentRow * cols + currentCol] =
103         currentWord;
104     ++currentCol;
105
106     if (isNewline)
107     {
108         if (currentCol != cols ||
109             locIdx != -1 && !isValidLocation(datas.fileDataArr[
110                 currentRow * cols + locIdx]) ||
111             timeIdx != -1 && !fast_stoull(datas.fileDataArr[
112                 currentRow * cols + timeIdx]))
113         {
114             --datas.rowSize;
115             currentCol = 0;
116             currentWord = &datas.fileData[i + 1];
117             continue;
118         }
119
120         datas.rows[currentRow] = &datas.fileDataArr[currentRow
121             * cols];
122         ++currentRow;
123         currentCol = 0;
124     }
125
126     currentWord = &datas.fileData[i + 1];
127 }
128
129 if (currentRow < datas.rowSize)
130 {
131     if (currentCol < cols)
132     {
133         datas.fileDataArr[currentRow * cols + currentCol] =
134             currentWord;
135     }
136     ++currentCol;
```

```
137
138     if (currentCol != cols ||
139         locIdx != -1 && !isValidLocation(datas.fileDataArr[
```

```
137         currentRow * cols + locIdx]) ||
138         timeIdx != -1 && !fast_stoull(datas.fileDataArr[
139             currentRow * cols + timeIdx]))
140     {
141         --datas.rowSize;
142     }
143     else
144     {
145         datas.rows[currentRow] = &datas.fileDataArr[currentRow *
146             cols];
147     }
148     return datas.rowSize;
149 }
```

Trong đó có phần xử lý data bị lỗi như: Thiếu hoặc thừa cột, Location không nằm trong danh sách thầy cho hoặc timestamp ≤ 0 , khi đó em sẽ bỏ qua và không thêm vào mảng rows nữa. Còn trường hợp bị trùng thì em vẫn để xử lý vì trùng thì có thể là do người dùng double click dẫn tới hệ thống lưu lại 2 dòng trùng nhau thì vẫn tính là data hợp lệ.

2.2. Tiền xử lý

Em đã viết hàm sort theo cột dùng std::sort của thư viện algorithm và hàm sort mảng Index dùng stable_sort để sort theo các cột giữ nguyên thứ tự timestamp:

Listing 3: Hàm sortByColumn và sortIdxArr

```
1 struct NumIdx
2 {
3     unsigned long long val;
4     int idx;
5 };
6
7 struct StrIdx
8 {
9     const char* val;
10    int idx;
11 };
12
```



```
13 void sortByColumn(DataArray& datas, const string& columnName,
14     bool isNum)
15 {
16     int colIdx = datas.getColumnIndex(columnName);
17     if (colIdx == -1)
18     {
19         cout << "Khong tim thay cot '" << columnName << "' de sap
20             xep.\n";
21         return;
22     }
23     if (isNum)
24     {
25         NumIdx* idxArr = new NumIdx[datas.rowSize];
26         for (int i = 0; i < datas.rowSize; ++i)
27         {
28             idxArr[i].val = fast_stoull(datas.rows[i][colIdx]);
29             idxArr[i].idx = i;
30         }
31         sort(idxArr, idxArr + datas.rowSize, [](const NumIdx& a,
32             const NumIdx& b)
33         {
34             return a.val < b.val;
35         });
36
37         char*** newRows = new char** [datas.rowSize];
38         for (int i = 0; i < datas.rowSize; ++i)
39         {
40             newRows[i] = datas.rows[idxArr[i].idx];
41         }
42
43         delete[] datas.rows;
44         datas.rows = newRows;
45         delete[] idxArr;
46     }
47     else
48     {
49         StrIdx* idxArr = new StrIdx[datas.rowSize];
50         for (int i = 0; i < datas.rowSize; ++i)
51         {
```

```
50     idxArr[i].val = datas.rows[i][colIdx];
51     idxArr[i].idx = i;
52 }
53 sort(idxArr, idxArr + datas.rowSize, [](const StrIdx& a,
54     const StrIdx& b)
55 {
56     return strcmp(a.val, b.val) < 0;
57 });
58
59 char*** newRows = new char** [datas.rowSize];
60 for (int i = 0; i < datas.rowSize; ++i)
61 {
62     newRows[i] = datas.rows[idxArr[i].idx];
63 }
64
65 delete[] datas.rows;
66 datas.rows = newRows;
67 delete[] idxArr;
68 }
69
70 StrIdx* sortIdxArr(const DataArray& datas, const string&
71     columnName)
72 {
73     int colIdx = datas.getColumnIndex(columnName);
74     if (colIdx == -1) return nullptr;
75
76     StrIdx* idxArr = new StrIdx[datas.rowSize];
77     for (int i = 0; i < datas.rowSize; ++i)
78     {
79         idxArr[i].val = datas.rows[i][colIdx];
80         idxArr[i].idx = i;
81     }
82
83     stable_sort(idxArr, idxArr + datas.rowSize, [](const StrIdx&
84         a, const StrIdx& b)
85     {
86         return strcmp(a.val, b.val) < 0;
```

```
87     return idxArr;  
88 }
```

Ở đây em tạo các mảng Index để sort nhằm tối ưu thời gian sắp xếp và coi như là tạo mảng tạm để tiện sử dụng cho các chức năng khai thác.

Dựa vào các hàm sort ở trên thì em đã tiền xử lí theo thứ tự là sẽ sort mảng gốc bằng hàm sortByColumn theo timestamp, sau đó sẽ chia luồng ra và sort các hàm cần cho các chức năng khai thác (cụ thể là 2 chức năng khai thác đầu trong yêu cầu giữa kì) để tối ưu thời gian tiền xử lí:

Listing 4: Tiền xử lí trong main

```
1 StrIdx* userIdxArr = nullptr;  
2 StrIdx* resourceIdxArr = nullptr;  
3  
4 int main()  
5 {  
6     cout << "\nDang_tien_xu_ly_(sap_xep_va_danh_chi_muc_song_  
7         song)\n";  
8  
9     sortByColumn(datas, "timestamp", true);  
10    thread t1([&]() { userIdxArr = sortIdxArr(datas, "user_id")  
11        ; });  
12    thread t2([&]() { resourceIdxArr = sortIdxArr(datas, "  
13        resource_id"); });  
14    t1.join();  
15    t2.join();  
16    cout << "\nTien_xu_ly_da_xong.\n";  
17 }
```

2.3. Thống kê top 10 tài nguyên

Ở phần này, em đã lấy mảng đã sort theo resource để đếm các resource nhanh hơn, sau khi đếm và đưa hết vào mảng resourceArr thì em đã dùng `stable_sort` để sort theo số lượt truy cập và không làm lẫn thứ tự, tức là nếu có nhiều resource đồng hạng 10 thì sẽ ưu tiên các resource có id đứng trước theo thứ tự alphabet.

Listing 5: Hàm `getTop10Resources`

```
1 void getTop10Resources(const DataArray& datas, StrIdx*
   resourceIdxArr, unsigned long long startTimestamp, unsigned
   long long endTimestamp)
2 {
3     int timeIdx = datas.getColumnIndex("timestamp");
4
5     if (!resourceIdxArr || timeIdx == -1)
6     {
7         cout << "Khong du thong tin de hien thi!\n";
8         return;
9     }
10
11     cout << "\nTop 10 Resources duoc truy xuat nhieu nhat tu " <<
        startTimestamp << " den " << endTimestamp << " la:\n";
12     StrIdx* resourceArr = new StrIdx[datas.rowSize];
13     int size = 0;
14     int idx = 0;
15     while (idx < datas.rowSize)
16     {
17         const char* resource = resourceIdxArr[idx].val;
18         if (!resource || resource[0] == '\0')
19         {
20             ++idx;
21             continue;
22         }
23
24         int cnt = 0;
25         while (idx < datas.rowSize && strcmp(resourceIdxArr[idx].
            val, resource) == 0)
26         {
27             unsigned long long timestamp = fast_stoull(datas.rows[
                resourceIdxArr[idx].idx][timeIdx]);
```

```
28     if (timestamp >= startTimestamp && timestamp <=
29         endTimeStamp)
30     {
31         ++cnt;
32     }
33     ++idx;
34     resourceArr[size].val = resource;
35     resourceArr[size].idx = cnt;
36     ++size;
37 }
38
39 if (size == 0)
40 {
41     cout << "Khong co tai nguyen nao duoc truy xuat trong
42         khoang thoi gian nay.\n";
43     delete[] resourceArr;
44     return;
45 }
46
47 stable_sort(resourceArr, resourceArr + size, [](const StrIdx&
48     a, const StrIdx& b)
49 {
50     return a.idx > b.idx;
51 });
52
53 for (int i = 0; i < ((size < 10) ? size : 10); ++i)
54 {
55     cout << i + 1 << ". " << resourceArr[i].val << ": " <<
56         resourceArr[i].idx << " luot truy cap\n";
57 }
58
59 delete[] resourceArr;
60 }
```

3. Cách sử dụng

Trước khi chạy file thực thi, copy file csv và paste vào cùng thư mục với file thực thi đó, đồng thời đổi tên file thành "data.csv". Sau đó mở file thực thi lên sẽ thấy hệ thống ghi rõ các bước như đọc file, tiền xử lý và cho người dùng chọn các chức năng đang hiện có:

1. **Hành trình Device → App → Resource của 1 User:** Hệ thống yêu cầu người dùng nhập **User ID**, thời gian bắt đầu và kết thúc. Sau đó in ra thông tin chuỗi hành trình khớp với điều kiện.
2. **Hành trình User → Device → App của 1 Resource:** Hệ thống yêu cầu người dùng nhập **Resource ID**, thời gian bắt đầu và kết thúc. Sau đó in ra thông tin chuỗi hành trình khớp với điều kiện.
3. **Top 10 Resource được truy xuất nhiều nhất:** Hệ thống yêu cầu nhập thời gian bắt đầu và kết thúc. Sau đó in ra 10 resource_id có số lượt truy cập cao nhất.
4. **Thoát và giải phóng bộ nhớ:** Hệ thống sẽ giải phóng các bộ nhớ động phát sinh trong lúc chạy và đóng chương trình.

4. Các khó khăn gặp phải và Hướng giải quyết

1. Giảm hiệu năng do std::string: Trong lần làm đầu tiên, em sử dụng std::string để lưu trữ dữ liệu, nhưng với mong muốn có thể tạo ra một hệ thống có thể chạy được với 1 triệu dòng để có thể tái sử dụng trong bài cuối kì thì em đã thử đọc file 1 triệu dòng và mất từ 30 đến 45 giây. Khi đó em chuyển dần sang việc dùng mảng const char* và cải tiến dần lên bằng việc chỉ đọc từng dòng vào đúng 1 mảng thì giờ đây hệ thống đã có 3 mảng để tăng tốc độ xử lý và tiện cho việc copy dữ liệu.
2. Tốc độ chậm khi thuật toán sắp xếp hoán vị những dòng dữ liệu chồng kèn: Ban đầu, hệ thống chạy các chức năng truy xuất hành trình bằng việc sắp xếp trên mảng dữ liệu lớn, khiến cho việc hoán đổi vị trí giữa 2 dòng mất rất nhiều thời gian vì phải hoán đổi từng thuộc tính. Vì thế em đã tham khảo hướng đi Sắp xếp gián tiếp qua chỉ mục (Sort by Index), khi đó chỉ cần tạo struct chứa giá trị cần sort và index của giá trị đó trong mảng lớn, khi đó mình chỉ cần sort mảng struct này thôi thì chỉ có 2 thuộc tính để hoán vị, và khi cần truy xuất thì lấy index của mảng nhỏ đưa cho mảng lớn để truy xuất thông tin.